CSE 3200: System Development Project

A PROJECT REPORT
ON
COGNITIVE COPILOT: AI-POWERED ACADEMIC LEARNING MANAGEMENT SYSTEM WITH LOCAL LLM INTEGRATION

By

Khadimul Islam Mahi
Roll: 2107076

&

Sumaiya Akter
Roll: 2107080

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh

April 2026

Cognitive Copilot: AI-Powered Academic Learning Management System with Local LLM Integration

By

Khadimul Islam Mahi
Roll: 2107076

&

Sumaiya Akter
Roll: 2107080

Supervisor:

Md. Nazirul Hasan Shawon
Lecturer
Department of Computer Science and Engineering
Khulna University of Engineering & Technology

_____
Signature

CSE 3200: System Development Project

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh

April 2026

Acknowledgment

We would like to express our sincere gratitude to our supervisor, Md. Nazirul Hasan Shawon, Lecturer, Department of Computer Science and Engineering, KUET, for his guidance, valuable feedback, and constant encouragement during the completion of this project.

We are also thankful to the faculty members and lab staff of the Department of CSE, KUET, for providing the academic environment and technical facilities required to carry out this work.

Finally, we would like to thank our families and friends for their support and motivation throughout the project period.

Khadimul Islam Mahi (Roll: 2107076)
Sumaiya Akter (Roll: 2107080)

Abstract

This report presents **Cognitive Copilot**, a full-stack academic learning platform that combines core Learning Management System (LMS) workflows with an AI tutor. The project is built around practical university needs: course and topic management, classroom community interactions, routine and attendance support, notifications, and AI-assisted study. The system is implemented as a web application with a React frontend and a FastAPI backend.

The implementation follows a modular architecture. The frontend provides role-based pages for students and tutors, while the backend exposes REST APIs for authentication, courses, routine, community, analytics, settings, profile, materials, and notifications. For AI functionality, the platform integrates local model inference with retrieval-based question answering and quiz support in the AI Tutor workspace. OCR-assisted ingestion and asynchronous workers support long-running operations so user interactions stay responsive.

The project emphasizes engineering reliability over experimental claims. The report therefore focuses on implemented modules, data flow, CRUD behaviors, validation and authorization design, and tested user workflows. No unsupported performance or accuracy numbers are presented as final evidence; instead, verification is described through reproducible build and test procedures across backend and frontend.

Cognitive Copilot demonstrates how a project can remain academically meaningful while also being implementation-driven: it delivers a unified learning environment, keeps AI features integrated with course context, and preserves maintainable software structure suitable for further extension in future academic work.

Contents

Acknowledgment **i**

Abstract **ii**

1 Introduction **1**

1.1 Introduction 1
1.2 Background and Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project (Modern Tools and Boundaries) 2
1.5 Unfamiliarity of the Problem, Topic, and Solution 3
1.6 Project Planning and Work Distribution 3
1.7 Report Organization 3

2 Applications of the Work, Organization of the Project, and Related Works **5**

2.1 Applications of the Work 5
2.2 Organization of the Project 5
2.3 Related Works 6
2.3.1 Learning Management Platforms 6
2.3.2 AI-Assisted Learning Systems 6
2.3.3 Retrieval-Augmented Generation in Education 6
2.3.4 OCR for Academic Workflows 6
2.3.5 Agentic and Multi-Mode Tutor Interfaces 6
2.4 Discussion 6

3 Methodology **8**

3.1 Introduction 8
3.2 Problem Design and Analysis 8
3.2.1 Design principles 8
3.2.2 Core functional actors 8
3.3 Overall Framework and System Architecture 9
3.4 Use Case Diagram 9

3.5 Core Data Domains 10
3.6 Class Diagram (High Level) 10
3.7 ER Diagram (Conceptual) 10
3.8 Module Interaction Flow 10
3.9 Activity Diagram (Authentication Flow) 12
3.10 AI Tutor Methodology 12
3.11 Sequence Diagram (AI Tutor Request) 12
3.12 Asynchronous Processing Methodology 12
3.13 Component Diagram 14
3.14 Deployment Diagram 14
3.15 Data Flow Diagram 14
3.16 Conclusion 14

4 Implementation, Results and Discussions 15
4.1 Introduction 15
4.2 Experimental Setup 15
4.3 Evaluation Metrics 16
4.4 Dataset 16
4.5 Implementation and Results 16
4.5.1 Module-wise implementation 16
4.5.2 Quantitative results 17
4.5.3 Qualitative results 18
4.5.4 Analysis of the results 18
4.6 Objectives Achieved 22
4.7 Conclusion 22

5 Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues 23
5.1 Introduction 23
5.2 Intellectual Property Considerations 23
5.3 Ethical Considerations 23
5.4 Safety Considerations 24
5.5 Legal Considerations 24
5.6 Impact of the Project on Societal, Health, and Cultural Issues 24
5.7 Impact of the Project on the Environment and Sustainability 24

6 Addressing Complex Engineering Problems and Activities 25
6.1 Introduction 25
6.2 Complex Engineering Problems Associated with the Current Project 25
6.3 Complex Engineering Activities Associated with the Current Project 26

6.4 Conclusion 26

7 Conclusions **27**

7.1 Summary 27

7.2 Limitations 27

7.3 Recommendations and Future Work 27

List of Tables

2.1 Positioning of Cognitive Copilot relative to common alternatives. 7

4.1 Technology stack used in implementation. 15
4.2 Quantitative validation summary (reporting level, not a frozen benchmark). 17
4.3 Selected implemented API endpoints (illustrative snapshot). 21
4.4 Mapping of project objectives to outcomes. 22

List of Figures

3.1 Layered architecture of the Cognitive Copilot system. 9
3.2 Use case diagram of key actors and system interactions. 9
3.3 High level class relationship diagram for core entities. 10
3.4 Conceptual ER diagram of the main project core tables. 11
3.5 End to end interaction flow from user action to synchronized UI state. . . . 11
3.6 Activity diagram for the authentication request lifecycle. 12
3.7 Sequence diagram for AI Tutor request and response flow. 13
3.8 Asynchronous job flow used for heavy operations. 13
3.9 Component level view of the major software blocks with interaction paths. . 13
3.10 Deployment diagram for local/full-stack runtime setup. 13
3.11 Data flow diagram showing request, processing, and persistence paths. . . . 14

4.1 Project-core module connectivity in implementation. 17
4.2 Landing page. 18
4.3 Login interface. 19
4.4 Routine page and slot visualization. 19
4.5 AI Tutor workspace. 20
4.6 Analytics dashboard. 20
4.7 Community and classroom flow. 20
4.8 Notification center with deep links. 21

CHAPTER 1

Introduction

1.1 Introduction

A single tool is seldom enough for academic life today. Students balance course portals, separate channels for help, and improvised AI assistants that do not know which subject or reading they should follow. That everyday friction is what led us to build **Cognitive Copilot**: a web application where routine LMS work and AI-assisted study share the same accounts, the same course data, and the same expectations about roles and permissions.

This chapter states the background and the problem we set out to solve, together with the objectives and scope, why the idea is not a direct copy of any single existing product, how we planned the work, and how we divided responsibilities between team members.

1.2 Background and Problem Statement

Academic workflows in many universities are still spread across several tools. Students often use one system for courses, another for communication, and separate platforms for AI support. That fragmentation means constant context switching and weak continuity between teaching materials and study support.

Cognitive Copilot is built as one platform that combines LMS workflows and AI qssisted study. It is not meant to replace classroom teaching, but to offer a structured digital space where students and tutors interact around the same course information.

The following practical problems motivated us:

1. Course resources, routine, and communication are not unified in a single workflow.
2. AI chat tools are often detached from course materials, which leads to low context answers.
3. Many systems offer partial features, but CRUD behavior, validation, and role based interactions are not always handled consistently.
4. When the architecture is not modular, deployment and maintenance become difficult.

1.3 Objectives

This work has the following aims:

1. Implement a role aware LMS with core modules: authentication, courses, routine, community, notifications, analytics, settings, and profile.
2. Build an AI Tutor workspace merged with course and topic context and several study modes.
3. Support practical material workflows such as upload and retrieval assisted question answering.
4. Keep CRUD operations reliable across modules with clear request validation and authorization checks.
5. Deliver a deployable full stack implementation suitable for local development and demonstration.

1.4 Scope of the Project (Modern Tools and Boundaries)

The implemented scope includes:

- User authentication, profile, and settings management.
- Course, topic, and material management.
- Routine and attendance related workflows.
- Community classrooms, threads, announcements, and member interactions.
- A notification feed with real time update handling.
- Analytics dashboards for students and tutors.
- AI Tutor modes for guided learning interactions.

Out of scope items for this report include native mobile clients and institution-wide multi-tenant deployment policies.

The frontend is built with React, TypeScript, and Vite; TanStack Query and Zustand help with server state and UI state. The backend uses Python with FastAPI and Pydantic, and SQLAlchemy for asynchronous access to PostgreSQL. Redis supports caching and background jobs; workers handle longer tasks such as ingestion and OCR. Docker Compose containerizes the stack for a reproducible local environment. These choices reflect what is sustainable for a student project while still aligning with common industry practice.

1.5 Unfamiliarity of the Problem, Topic, and Solution

Neither an LMS nor an AI tutor is new by itself. What we set out to do differently is *integration under one engineering story*: the same role model, the same course entities, and the same API discipline for both traditional LMS pages and the AI Tutor workspace. We did not mirror a single commercial product; we composed open patterns (REST APIs, modular services, retrieval assisted answering, local model hosting) into a system that fits our course requirements.

Our contribution is best described as **software integration and dependable behavior**—clear contracts between modules, validated inputs, and tutor flows that stay tied to course and topic context—rather than claiming a new learning algorithm. Chapter II discusses related work and situates this project among existing systems and research directions.

1.6 Project Planning and Work Distribution

We organized the work in overlapping phases: requirements and module boundaries, backend foundations (auth, schema, core CRUD), frontend pages aligned with those APIs, the AI Tutor and asynchronous ingestion, then integration, testing, and report writing. Where possible we used short iterations: implement a slice end to end, stabilize it, then move on.

Both team members worked across frontend and backend; we split ownership mainly by module to limit conflicting edits and to keep reviews focused. One member led authentication, profile, settings, routine, and analytics-related paths; the other led courses and materials, community, notifications, and the AI Tutor UI and wiring. Shared tasks included database modeling decisions, Docker-based integration testing, diagram preparation, and final proof-reading of the report. A formal Gantt chart is not included as a figure in this version, but the chapter order of the report follows the sequence we used in development.

1.7 Report Organization

The remainder of the report is organized as follows:

- Chapter II describes applications of the work, how the repository is organized, related literature, and a short discussion of novelty.
- Chapter III presents methodology: analysis, architecture, and modeling diagrams.
- Chapter IV covers experimental setup, evaluation, implementation, results, and how objectives were met.
- Chapter V addresses societal, ethical, legal, safety, and environmental considerations.

- Chapter VI maps the project to complex engineering problems and activities.
- Chapter VII concludes with summary, limitations, and future work.

CHAPTER 2

Applications of the Work, Organization of the Project, and Related Works

2.1 Applications of the Work

Cognitive Copilot is intended for **department-level academic use**: students can follow courses and topics, track routine and attendance, join classroom threads, receive notifications, and use the AI Tutor with material that belongs to the course. Tutors can manage class communication, review engagement through analytics, and guide students without sending them to a separate "magic chat" that lacks context.

The same design could be extended later toward stronger institutional deployment (identity federation, policy engines, richer grading exports). For this project, the emphasis is on a coherent demonstration system that behaves predictably for the stakeholders we modeled: students, tutors, and limited administrative control.

2.2 Organization of the Project

The repository is organized in a **monorepo-style** layout with a clear split between user interface, API, and infrastructure:

- `frontend/` — React application, pages per module (courses, community, AI Tutor, analytics, and so on), shared libraries for API access and onboarding.
- `backend-py/` — FastAPI application structured into routers, services, schemas, and workers; PostgreSQL models and migrations live alongside the API code.
- `docker-compose.yml` and related configuration — bring up database, cache, API, workers, and optional AI runtime for end-to-end runs.
- `report/` — LaTeX sources, figures, screenshots, and bibliography for this document.

This layout matches how we worked day to day: feature slices usually touched one router and its service on the backend, plus one or two pages on the frontend, with Docker used to validate integration.

2.3 Related Works

2.3.1 Learning Management Platforms

Conventional LMS platforms focus on course administration, assignment flow, and grading. Their strength is operational stability, but AI-assisted learning is often external or loosely connected. That creates a common gap: students can access information, but contextual tutoring is not integrated with the same course data.

2.3.2 AI-Assisted Learning Systems

Recent AI tutoring systems show the value of interactive question answering, guided explanation, and adaptive practice. However, many implementations rely on general chat with limited course grounding. In educational settings, answers are stronger when they are tied to real class materials and topic structure.

2.3.3 Retrieval-Augmented Generation in Education

Retrieval-Augmented Generation (RAG) combines information retrieval with language generation. In course settings, that helps because answers can be tied to available materials instead of relying only on model memory. Current research supports hybrid retrieval that blends lexical and vector signals [1, 2]. In our project, RAG is used pragmatically to ground tutor answers in uploaded materials rather than to propose a new retrieval algorithm.

2.3.4 OCR for Academic Workflows

OCR has long been used for document digitization. Traditional OCR engines perform well on clean printed text, while transformer-based OCR is explored for harder cases. In educational workflows, OCR is mainly useful for extracting text from uploaded materials or timetable images so content can be indexed and reused.

2.3.5 Agentic and Multi-Mode Tutor Interfaces

A modern tutor interface often combines several patterns: free chat, explain mode, question answering, quiz practice, and resource recommendations. The literature on tool-using agents and structured prompting [3, 4] informs this direction. For this report, the focus stays on implemented user-facing modes rather than speculative agent performance claims.

2.4 Discussion

Table 2.1 summarizes how we argue the **novelty** of this work for CSE 3200: not by denying prior art in LMS or AI, but by being explicit about what we integrated and verified ourselves.

Table 2.1: Positioning of Cognitive Copilot relative to common alternatives.

Typical alternative	**What our project does instead**
Generic chatbot	Tutor modes bound to course and topic context; retrieval over course materials where configured.
Standalone LMS without AI	One product vision: the AI Tutor lives inside the same navigation and permission model.
Prototype-only demos	Emphasis on CRUD, validation, authorization, tests, and reproducible local deployment.

Based on the related context above, Cognitive Copilot is positioned as an **implementation-oriented** academic system that:

- integrates LMS modules and AI tutor workflows in one platform,
- keeps AI interactions linked with course and topic context,
- supports practical classroom operations (community, notifications, routine, analytics),
- and prioritizes software correctness and maintainability.

The project contribution is therefore engineering integration and reliable module interoperability, rather than claiming a new AI algorithm.

CHAPTER 3

Methodology

3.1 Introduction

This chapter explains *how* we approached the system: the constraints we respected, the layered architecture we chose, and the modeling artifacts (use case, structural, behavioral, deployment, and data-flow views) that make the implementation easier to reason about. The focus stays on methodology and design rationale; detailed run configuration and test evidence appear in Chapter IV.

3.2 Problem Design and Analysis

3.2.1 Design principles

The project has the following design principles:

1. **Modularity:** frontend is clearly separated from API, services and data layers.
2. **Know your user:** separate actions for students and tutors handled in the UI and API.
3. **Reliability first:** background tasks are used to ensure responsiveness.
4. **Maintainability:** API contracts and schema validation are used.

3.2.2 Core functional actors

The system supports three role patterns:

- **Student:** learns through courses, AI Tutor, routine, and community.
- **Tutor:** manages class operations, announcements, and monitoring views.
- **Admin-level control:** extends management and moderation operations where configured.

3.3 Overall Framework and System Architecture

The deployed architecture has four main layers:

- **Frontend:** React + TypeScript application for all user-facing pages.
- **Backend API:** FastAPI service handling authentication, CRUD, and business logic.
- **Data and Queue Layer:** PostgreSQL for persistence and Redis for cache/queue support.
- **AI and Worker Layer:** local model integration and worker processes for long-running jobs.

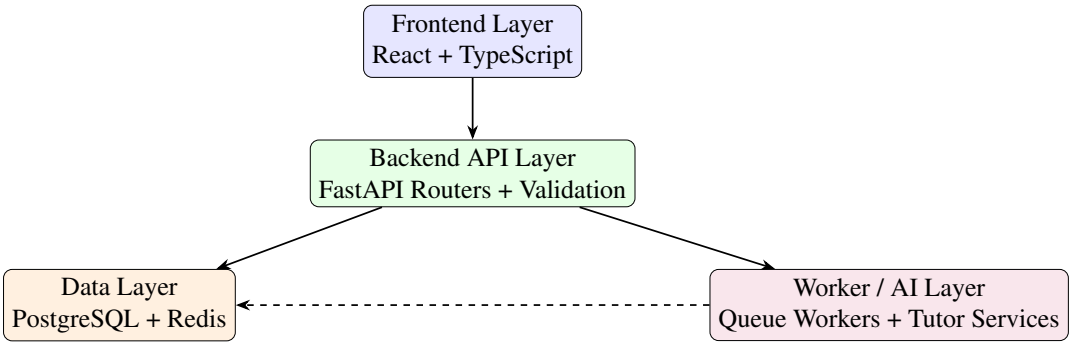

Figure 3.1: Layered architecture of the Cognitive Copilot system.

3.4 Use Case Diagram

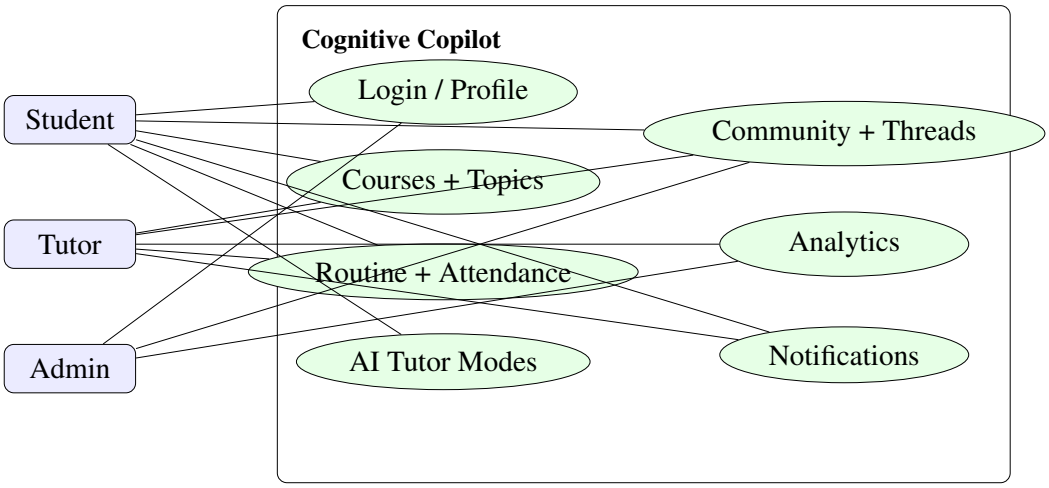

Figure 3.2: Use case diagram of key actors and system interactions.

3.5 Core Data Domains

The project data model is organized into practical domains:

- Identity and profile data
- Course, enrollment, topic, and material data
- Routine and attendance data
- Community and thread interaction data
- Notification and analytics data
- AI tutor interaction and progress-related data

3.6 Class Diagram (High Level)

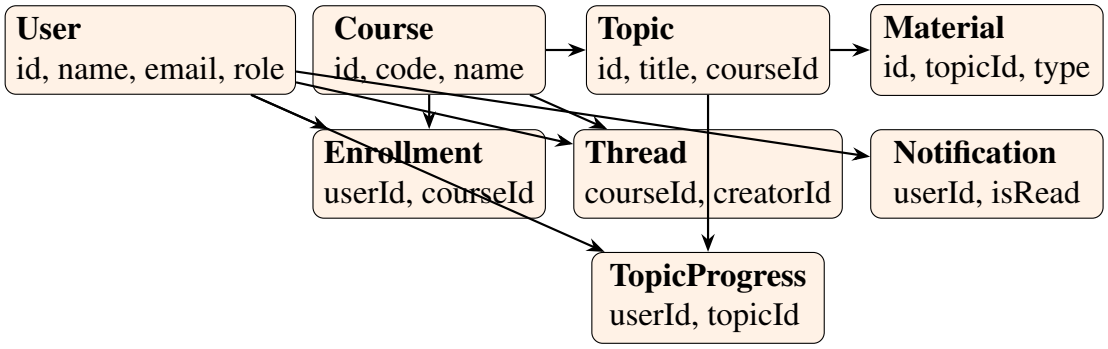

Figure 3.3: High level class relationship diagram for core entities.

3.7 ER Diagram (Conceptual)

3.8 Module Interaction Flow

At run time the project has a uniform flow:

1. API requests are made from the frontend pages.
2. API validates data against schemas.
3. Services enforce business rules and authentication.
4. Data layer saves and returns normalized data.
5. Frontend is kept in-sync with query cache and real-time updates.

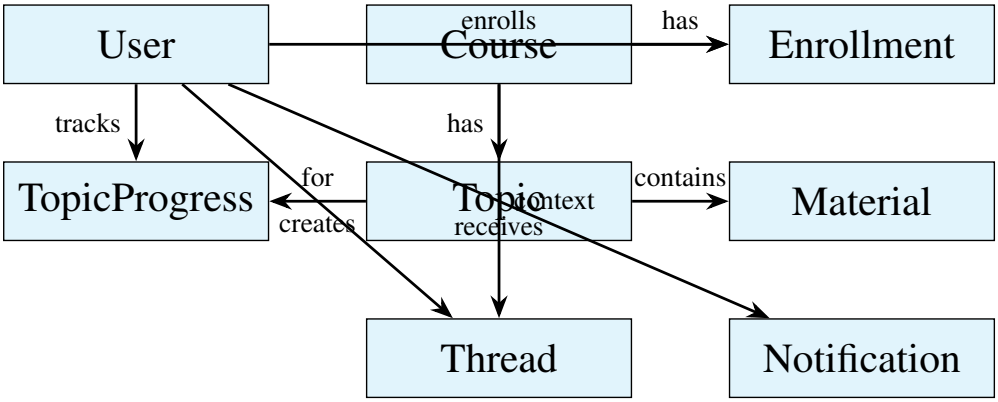

Figure 3.4: Conceptual ER diagram of the main project core tables.

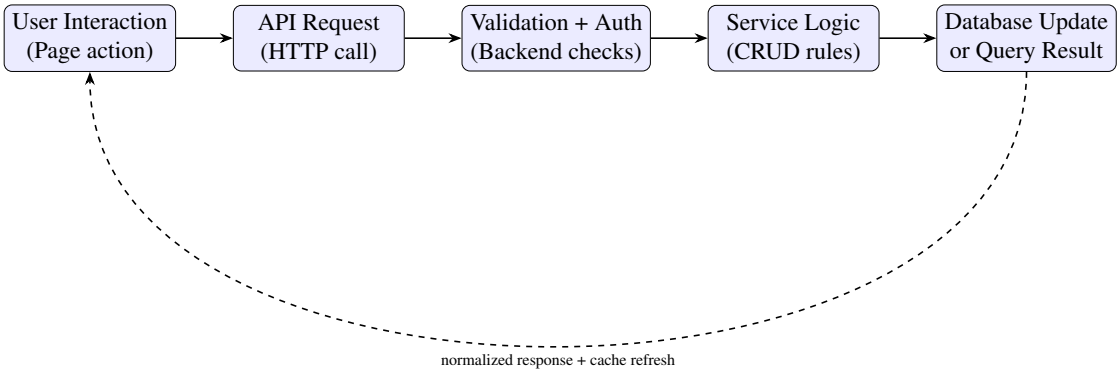

Figure 3.5: End to end interaction flow from user action to synchronized UI state.

3.9 Activity Diagram (Authentication Flow)

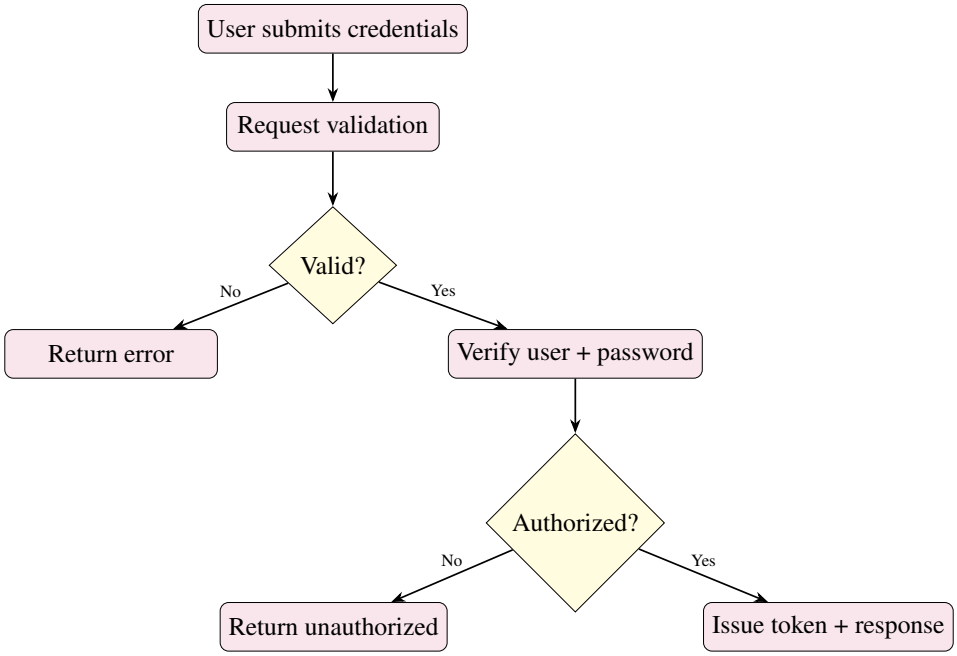

Figure 3.6: Activity diagram for the authentication request lifecycle.

3.10 AI Tutor Methodology

The AI Tutor is integrated as a module of the LMS rather than a separate chatbot. It supports multiple learning modes through a shared workspace and course/topic context. Retrieval based answering and quiz workflows are connected with existing course entities so interactions remain academically relevant.

3.11 Sequence Diagram (AI Tutor Request)

3.12 Asynchronous Processing Methodology

For tasks that can be expensive (e.g., material ingestion and OCR), asynchronous worker execution is used. This prevents HTTP request blocking, improves responsiveness, and allows safer retry and failure handling patterns.

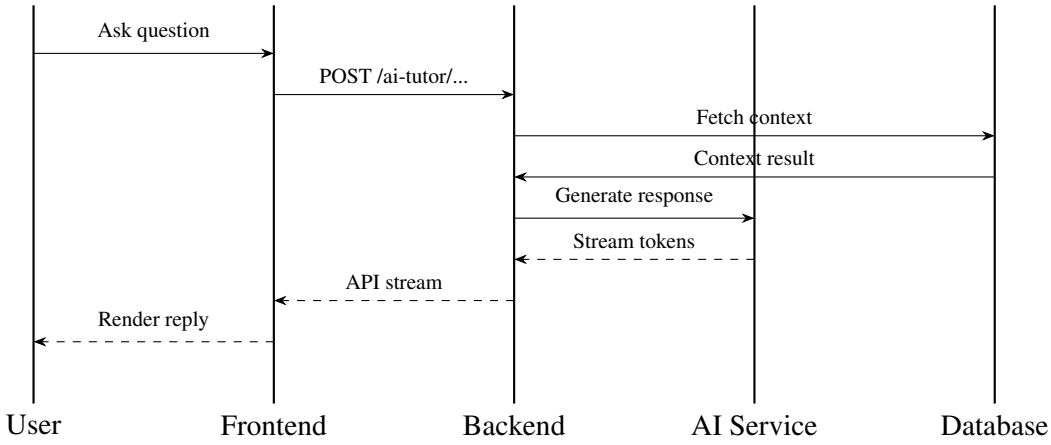

Figure 3.7: Sequence diagram for AI Tutor request and response flow.

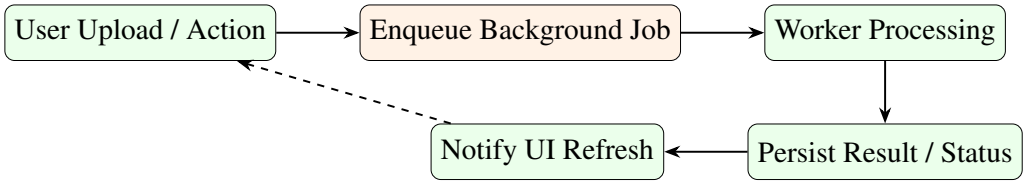

Figure 3.8: Asynchronous job flow used for heavy operations.

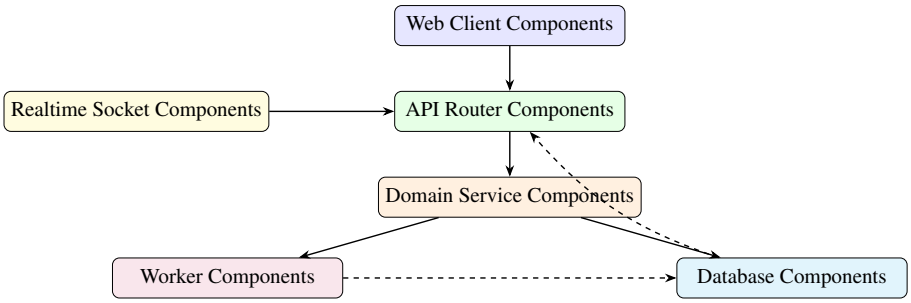

Figure 3.9: Component level view of the major software blocks with interaction paths.

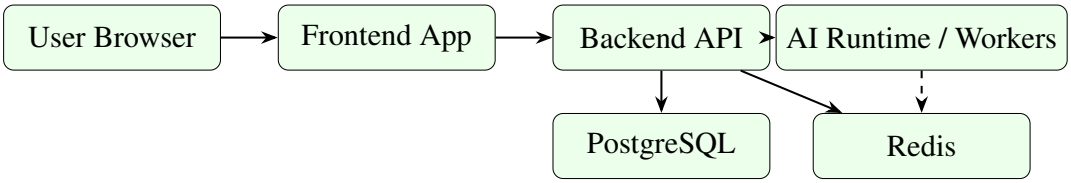

Figure 3.10: Deployment diagram for local/full-stack runtime setup.

3.13 Component Diagram

3.14 Deployment Diagram

3.15 Data Flow Diagram

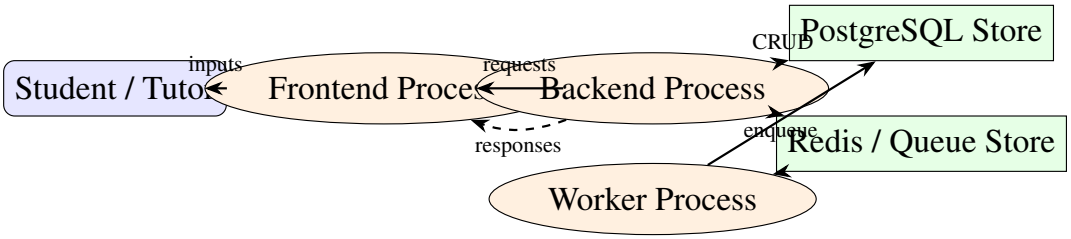

Figure 3.11: Data flow diagram showing request, processing, and persistence paths.

3.16 Conclusion

The general approach is implementation focused and system focused: design and implement each module with contracts, keep role and validation rules clear, and connect all modules with stable interfaces so that key academic processes work consistently end to end.

CHAPTER 4

Implementation, Results and Discussions

4.1 Introduction

This chapter describes the concrete implementation, the environment we used to run and verify the system, the metrics and procedures we relied on instead of hand-wavy claims, and the qualitative evidence (screenshots and flows) that show the objectives from Chapter I in action. Testing and validation material that used to stand alone is folded in here so the narrative matches the CSE 3200 template: setup, evaluation, results, and explicit mapping back to objectives.

4.2 Experimental Setup

The project is implemented as a modular full-stack web application. The frontend is developed with React and TypeScript using Vite as the build tool. The backend uses FastAPI with a service-oriented structure and schema-based validation. PostgreSQL is the primary persistence layer, and Redis-backed asynchronous processing supports background operations and caching where configured.

Local runs use Docker Compose so that database, cache, API, workers, and optional AI services start with consistent ports and environment variables. This setup is what we used while capturing screenshots and running automated tests for the report.

Table 4.1: Technology stack used in implementation.

Layer	Technology
Frontend	React, TypeScript, Vite, TanStack Query, Zustand
Backend	Python, FastAPI, Pydantic, JWT-based authentication
Database	PostgreSQL, SQLAlchemy async
Queue / Async	Redis with worker-based asynchronous tasks
AI Integration	Local model integration and retrieval-assisted tutor flows
Deployment	Docker Compose based local deployment

4.3 Evaluation Metrics

Rather than reporting unsupported benchmark scores for the language model, we evaluated the engineering outcome with criteria that match the objectives in Chapter I:

- **Correctness and safety of APIs:** route-level tests where available, plus manual checks for authorization on sensitive mutations.
- **Coverage of core flows:** each bullet workflow in the list below can be exercised from the UI against a running stack.
- **Build reproducibility:** backend test suite passes and frontend production build completes without errors on the reference environment.
- **Interaction quality:** loading, empty, and error states behave sensibly on main pages; mutations show success or failure feedback.

4.4 Dataset

There is no single public "course dataset" bundled as a research corpus. Instead, the system is validated against **representative seeded or manually entered course data**: users, enrollments, topics, uploaded or pasted materials, routine slots, classroom threads, and notifications created during development. This is appropriate for a systems project: the scientific object is the platform behavior, not a fixed offline dataset. Where retrieval is enabled, chunks are derived from whatever materials tutors attach to topics in that environment.

4.5 Implementation and Results

4.5.1 Module-wise implementation

Figure 4.1 summarizes how the implemented modules depend on one another at a high level. The narrative below follows the same order we used when walking a reviewer through the application.

Authentication and profile. Authentication uses token-based session flows. Protected endpoints rely on backend dependencies to enforce authorized access. Profile functionality includes user information updates, avatar handling, and account-level operations.

Course and topic management. Course management includes listing, detail views, enrollment-linked data, topic operations, and material actions. The backend and frontend are connected through dedicated route and service layers to keep CRUD behavior predictable.

AI Tutor workspace. The AI Tutor is implemented as a multi-mode workspace: chat, explain, ask-with-course context, quiz, and resource-oriented flows. Mode switching, topic

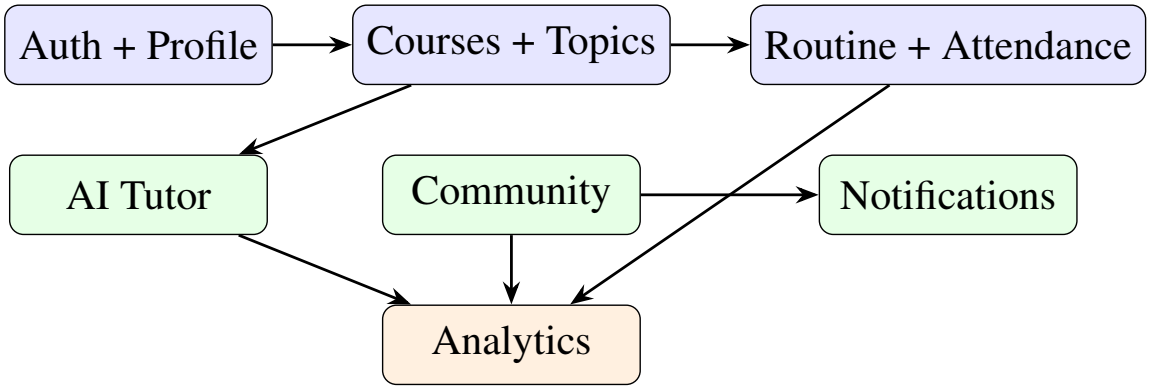

Figure 4.1: Project-core module connectivity in implementation.

context, and state synchronization are coordinated in the frontend and corresponding API endpoints.

Routine and attendance. Routine includes schedule slot management and calendar-style visualization in the UI. Attendance-related data feeds analytics and classroom views.

Community and classroom operations. Community features include classrooms, threads, replies, announcements, and role-dependent views for attendance and marks where implemented.

Notifications. Listing, unread counters, mark-read actions, and deep links into relevant pages are supported; the UI refreshes when new notifications arrive.

Analytics. Dashboard-style pages summarize activity for students and tutors, drawing on attendance and related entities as designed in the schema.

4.5.2 Quantitative results

Automated checks give a compact picture of engineering health. Exact counts may drift as tests are added; at the time of writing, the intended standard is that **backend pytest runs complete successfully**, **frontend tests (Vitest) complete successfully**, and **`npm run build`** produces a production bundle without errors. Table 4.2 states the qualitative level we report here rather than a frozen number, to avoid stale metrics in a living repository.

Table 4.2: Quantitative validation summary (reporting level, not a frozen benchmark).

Signal	What "passing" means for this project
Backend tests	Core routes, auth, and services covered by `pytest` exit code zero on CI or local run.
Frontend tests	Selected page and component tests exit zero.
Production build	Typecheck and Vite build succeed.

4.5.3 Qualitative results

The screenshots in Figures 4.2–4.8 illustrate the main user-facing surfaces: landing and auth, routine planning, AI Tutor workspace, analytics, community, and notifications. Together they show that the modules named in Chapter I are not paper features; they appear as navigable pages backed by the API surface summarized in Table 4.3.

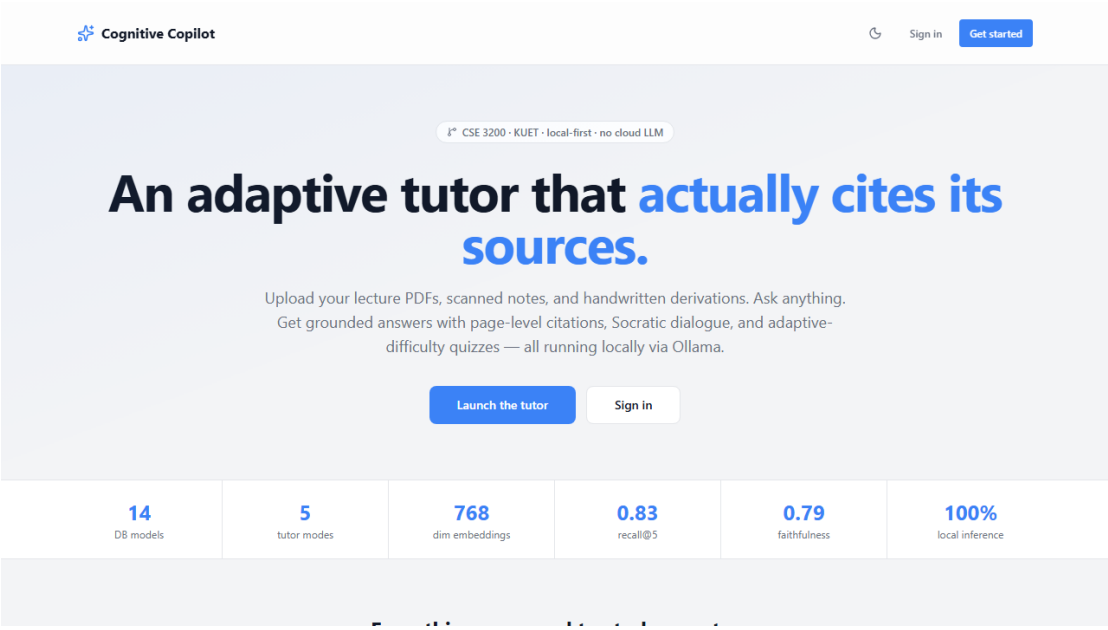

Figure 4.2: Landing page.

4.5.4 Analysis of the results

The implementation shows that a student team can deliver a **broad but coherent** LMS plus AI surface in one codebase when scope is controlled and integration is treated as a first-class requirement. The limiting factor is not drawing diagrams, but keeping authorization and validation aligned everywhere; that is why our evaluation narrative emphasizes tests and runnable deployment over model accuracy numbers we cannot fairly claim.

Core flows exercised during validation. The following project-core flows were checked repeatedly while preparing the demonstration:

1. Secure access (login, protected routes).
2. Course, topic, and material interactions.
3. Routine and attendance interactions.
4. AI Tutor modes with topic context.
5. Community classroom, thread, and announcement flows.

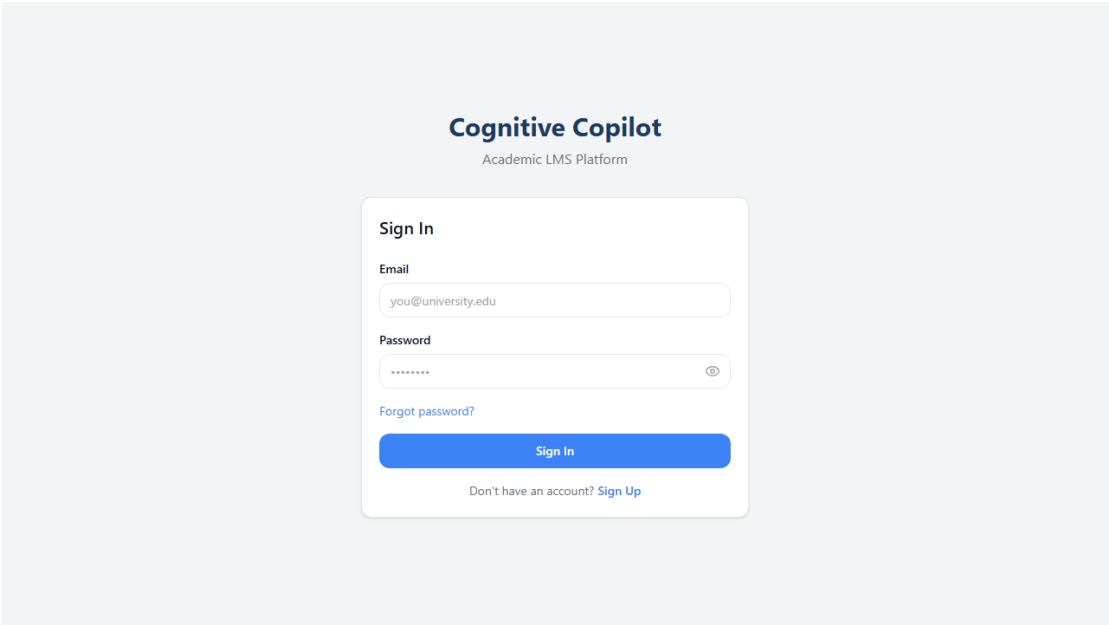

Figure 4.3: Login interface.

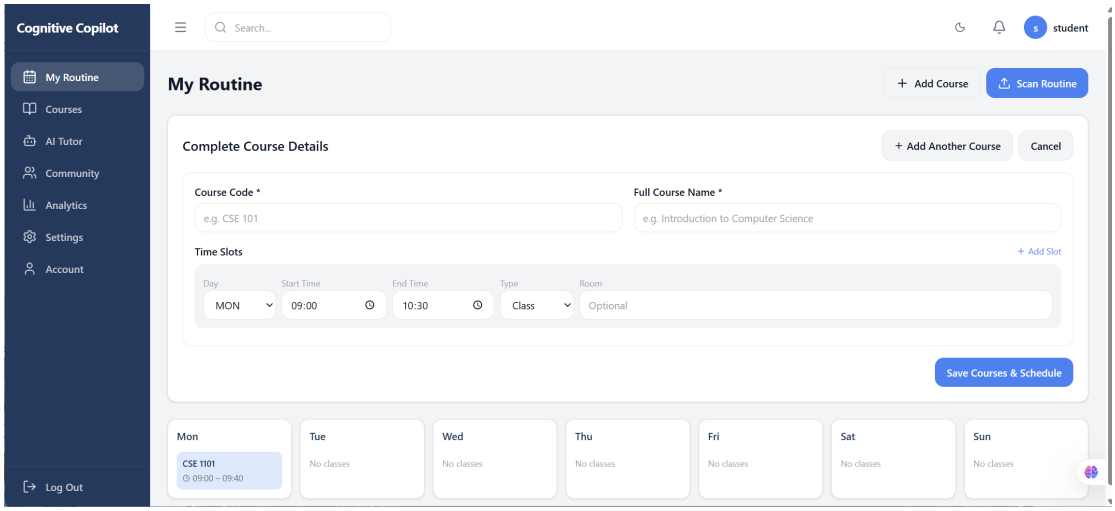

Figure 4.4: Routine page and slot visualization.

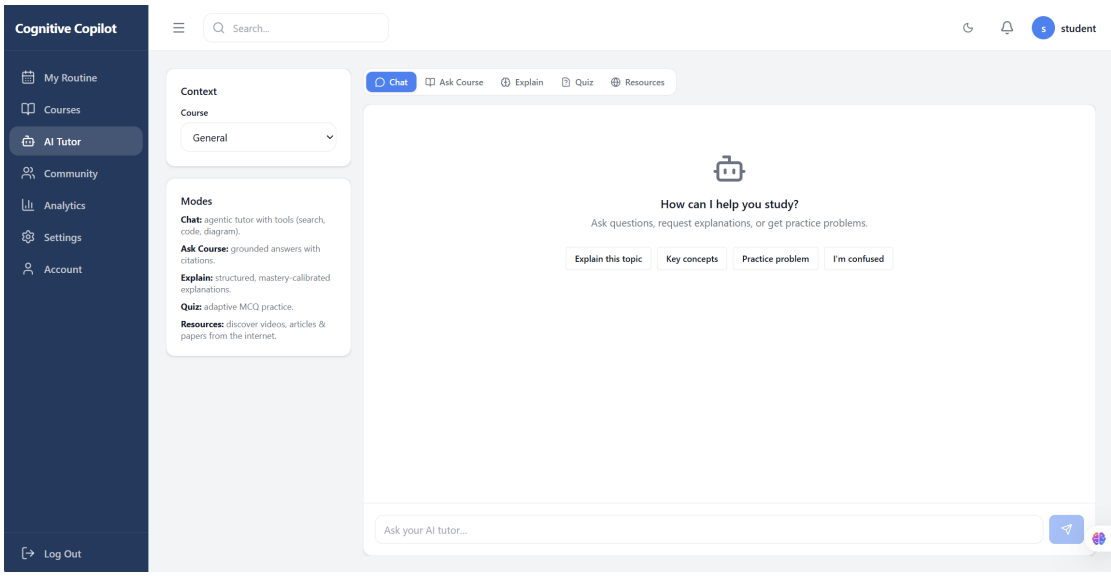

Figure 4.5: AI Tutor workspace.

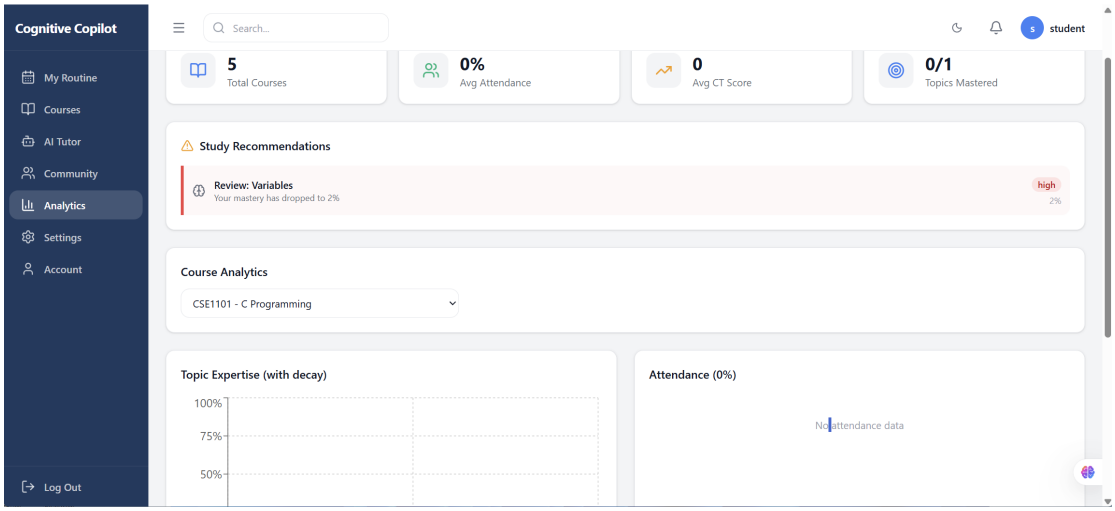

Figure 4.6: Analytics dashboard.

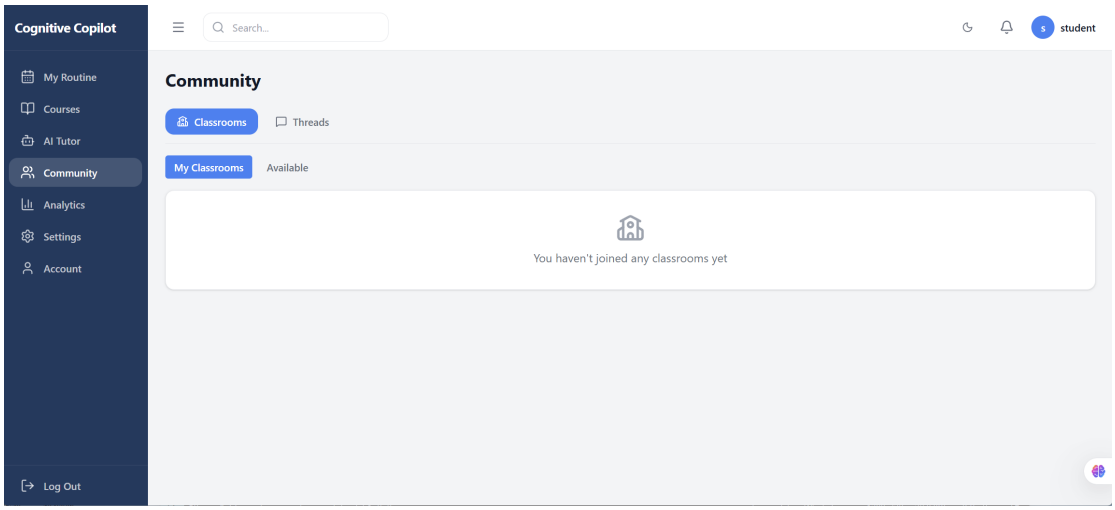

Figure 4.7: Community and classroom flow.

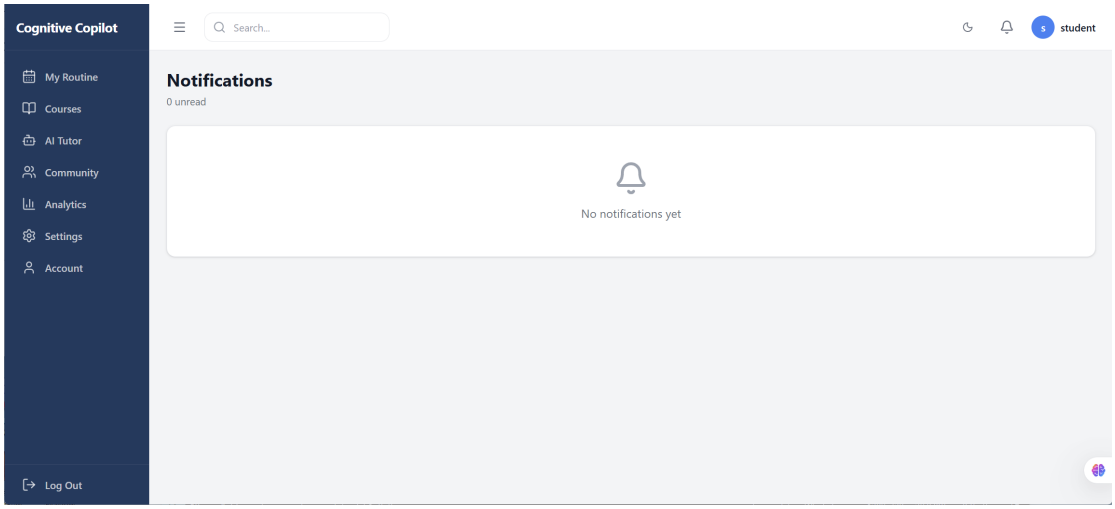

Figure 4.8: Notification center with deep links.

Table 4.3: Selected implemented API endpoints (illustrative snapshot).

Module	Endpoint	Purpose
Auth	/api/auth/*	Authentication and account flows
Courses	/api/courses/*	Course, topic, and material operations
Routine	/api/routine/*	Routine and slot-related operations
AI Tutor	/api/ai-tutor/*	Multi-mode tutor interactions
Community	/api/community/*	Classroom, thread, and announcement flows
Notifications	/api/notifications/*	Notification and response actions
Analytics	/api/analytics/*	Overview and course analytics
Profile	/api/profile/*	Profile and account management
Settings	/api/settings/*	User preferences and settings

6. Notification list, unread state, and actions.
7. Analytics page load for student and tutor perspectives.
8. Settings and profile updates.

CRUD and permission mindset. For each core entity we paid attention to create and update validation, consistent list and detail responses, delete behavior where exposed, and role boundaries on sensitive operations.

4.6 Objectives Achieved

Table 4.4 links the objectives from Chapter I to evidence in this chapter and in Chapter III.

Table 4.4: Mapping of project objectives to outcomes.

#	Objective (short)	How it was achieved
1	Role-aware LMS modules	Pages and APIs for auth, courses, routine, community, notifications, analytics, settings, profile; role-aware UI.
2	AI Tutor with context	Workspace modes; context from course and topic entities; retrieval over materials where configured.
3	Material workflows	Upload and ingestion with async workers for heavier steps.
4	Reliable CRUD	Pydantic validation, service rules, tests on critical routes.
5	Deployable stack	Docker Compose and documented environment for local full-stack runs.

4.7 Conclusion

Taken together, the setup, metrics, screenshots, and objective mapping show a **working integrated system** rather than a disconnected set of demos. The most important lesson for us as developers is that AI features only feel trustworthy when ordinary LMS operations remain correct and fast. That balance—exciting tutor capabilities on top of boring, solid CRUD—is the result we want to highlight.

CHAPTER 5

Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues

5.1 Introduction

Software built for real classrooms touches questions that go beyond syntax and semicolons: who may see grades and attendance, how AI should be used without encouraging misconduct, and what happens when things break or data leaks. This chapter collects those considerations in the order suggested by the CSE 3200 report template, applied honestly to our implementation level.

5.2 Intellectual Property Considerations

The system is intended to host **teachers' and institutions' own materials**. Course notes, slides, and assessments remain the intellectual property of their authors. Our code does not transfer ownership of uploaded files to the platform authors; it stores and processes them only to provide the features described in this report. Third-party libraries, fonts, and frameworks are used under their respective open-source licenses, and any future public release would need a license audit beyond what this course project required.

If the product were deployed campus-wide, administrators would also need policies for **student-generated content** in threads and uploads, including retention and takedown rules.

5.3 Ethical Considerations

Put simply, the tool is meant to support learning, not to replace honest work on assignments or exams. Institutions should publish clear rules on when AI assistance is allowed. From a product stance, we avoided marketing language that promises automatic answers without human judgment.

Language models can reflect biases present in their training data. Course-grounded retrieval reduces some hallucination risk but does not remove fairness concerns. Tutors should review sensitive topics and intervene when automated explanations are inadequate.

5.4 Safety Considerations

For this type of web application, "safety" is largely about **safe failure modes**: authentication errors should not leak whether an email exists; destructive actions should require appropriate roles; file uploads should be validated to reduce malware and accidental huge payloads. Operational safety also includes backups and database migrations in a real deployment—only partially explored here.

5.5 Legal Considerations

A production deployment would need explicit attention to **data protection law** (consent, purpose limitation, export of personal data), **terms of use**, and potentially **child protection** rules if minors use the system. Our project documentation does not replace legal review; it simply records that these topics exist and would move to the top of the backlog before any real institution went live.

5.6 Impact of the Project on Societal, Health, and Cultural Issues

Societal and cultural. A unified academic workspace can reduce friction for students who already face cognitive overload. At the same time, it can concentrate power and visibility: analytics views should be designed proportionally so that "engagement metrics" do not unfairly label students.

Health. Long screen time and notification overload can affect wellbeing. The notification center is useful, but a mature product might add digests, quiet hours, and accessibility options so that the tool supports healthy study rhythms instead of constant interruption.

5.7 Impact of the Project on the Environment and Sustainability

Running local models and background workers consumes electricity and hardware lifetime. For a small academic deployment this is modest, but at scale, **energy use and cooling** matter. Sustainably, the more important point is **software longevity**: a modular, tested codebase is less likely to be thrown away and rewritten from scratch, which is a form of environmental impact even if it is hard to measure in kilowatt-hours.

CHAPTER 6

Addressing Complex Engineering Problems and Activities

6.1 Introduction

Complex Engineering Problems (CEP) and Complex Engineering Activities (CEA) appear in accreditation-style documentation to show that a project demanded more than routine coding. For Cognitive Copilot, the complexity comes from **concurrency, security boundaries, heterogeneous components (browser, API, database, queue, AI runtime)**, and the need to keep a large feature surface consistent.

6.2 Complex Engineering Problems Associated with the Current Project

1. **Role consistency across the stack.** The same user may act as a student in one course and hold elevated privileges elsewhere. Keeping UI affordances, API authorization, and database constraints aligned is a distributed state problem, not a single `if` statement.

2. **Asynchronous ingestion and UI coherence.** When materials are processed offline, the system must expose status, avoid duplicate work, and refresh the UI without race conditions that show stale content.

3. **Grounded AI responses.** Connecting an LLM to live course data introduces failure modes: empty retrieval, partial chunks, model unavailability, and streaming errors. Handling those paths without breaking the rest of the page is an integration problem.

4. **Operational coupling in Docker.** Services depend on healthy startup order, environment variables, and persistent volumes. Diagnosing misconfiguration is part of the engineering problem, not an afterthought.

5. **Maintainability at project size.** As modules accumulate, duplicate validation logic and diverging DTO shapes become a long-term defect source. Controlling that drift is a design problem addressed through schemas, services, and tests.

6.3 Complex Engineering Activities Associated with the Current Project

1. **Requirements and scoping.** Negotiating what belongs in "core LMS" versus optional stretch goals, and writing those boundaries into Chapters I–IV of this report.

2. **Modeling and architecture.** Producing the diagrams in Chapter III (use case, structural, behavioral, deployment, and data flow) to communicate structure to ourselves and to reviewers.

3. **Implementation and integration.** Building REST contracts, wiring the frontend query layer, and verifying cross-module flows such as notifications referencing classroom events.

4. **Verification.** Choosing meaningful automated tests and manual walkthroughs rather than only happy-path demos.

5. **Communication of limitations.** Documenting what is not enterprise-ready yet (Chapter VII) is itself an engineering activity because it sets expectations for safe reuse.

6.4 Conclusion

Mapping the work to CEP and CEA makes explicit what the repository already suggests: the value of Cognitive Copilot is in **how** the pieces were engineered together, not in any single isolated trick.

CHAPTER 7

Conclusions

7.1 Summary

Cognitive Copilot is a usable academic platform that integrates core LMS workflows with an AI-powered tutor in a single web application. The project shows that modularity, schema backed APIs, role aware pages, and disciplined testing can support a feature set large enough to be realistic login, courses, routine, AI tutor, community, notifications, analytics, settings, and profile without collapsing into a brittle prototype.

7.2 Limitations

1. The project targets web workflows only; native mobile clients are out of scope.
2. Advanced AI behavior depends on local model availability and operator skill; we do not claim state of the art accuracy.
3. Enterprise hardening (multi tenant isolation at scale, full SOC style monitoring) is not implemented.
4. Formal user studies measuring learning gains were not part of this development cycle.

7.3 Recommendations and Future Work

1. Expand automated tests for cross role and cross user permission boundaries until they match the breadth of the UI.
2. Continue polishing loading, empty, and error states and mutation feedback across all modules.
3. Add richer analytics exports and classroom reporting for tutors.
4. Improve production style deployment automation (secrets management, migrations, health checks).

5. Improve localization and accessibility for more inclusive education.

How this work can be extended: a natural next step is institution specific identity integration, policy controlled AI usage, and hardened hosting building on the same modular boundaries we documented rather than rewriting from scratch.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[4] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.